Plague Containment

A Grid-Based Epidemic Strategy Game with Greedy & Backtracking Algorithms

Nelson David Molina Ramos · Brayan Estiven Aguirre Aristizabal · Juan Sneyder Méndez Gil

Computer Engineering Program — Universidad Distrital Francisco José de Caldas — Bogotá, Colombia

COMPUTER SCIENCES I · SEMESTER 2026-I · TEAM 16

1 Problem & Motivation

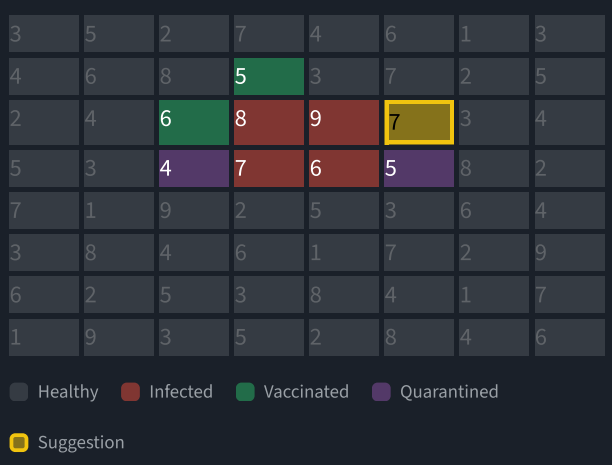
Epidemic containment requires **rapid local decisions** (which area to protect next) combined with **global planning** (how to isolate the infected zone without fragmenting healthy regions).

Plague Containment models this as a turn-based strategy game on an **8×8 city grid**, where each district carries a risk level (1–10) and the plague spreads every turn.

District States

State	Meaning
Healthy	Not yet affected
Infected	Has plague, spreads to neighbors
Vaccinated	Immune, blocks spread
Quarantined	Locked down for containment

Game Grid Example



2 Algorithm Design

Greedy Vaccination

Recommends the **highest-risk healthy neighbor** of the infected zone for immediate vaccination.

1. Scan all infected districts and collect their healthy neighbors (no duplicates)
2. Select the candidate with the highest risk level
3. Mark it as vaccinated and return the recommendation

> **Why greedy?** The most urgent threat is always the highest-risk unprotected neighbor. Local optimality aligns with the immediate game objective.

Backtracking Quarantine

Finds a set of districts to quarantine that **fully encloses** the infected zone **without isolating** any healthy area.

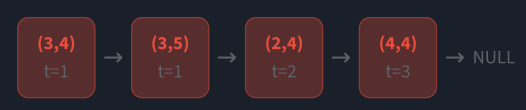
1. Identify all healthy border districts adjacent to infected cells
2. For each candidate: try quarantining it, then check connectivity
3. If connectivity breaks → **backtrack** (undo) and try next option
4. If full enclosure achieved with connectivity → return solution

> **Why backtracking?** A greedy approach could accidentally isolate a healthy region. Backtracking explores all valid combinations and undoes bad choices.

3 Data Structures (C++)

Linked List — Infection Chain

Stores the **chronological history** of plague spread. Each node contains the district position and the turn it was infected.



Why linked list? Infection grows dynamically — we don't know how many districts will be infected. Insertions at the tail are O(1).

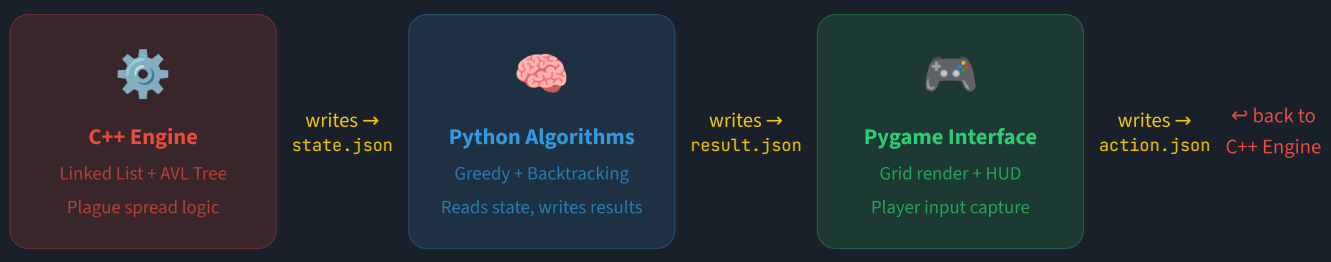
AVL Tree — District Risk Index

Stores districts **ordered by risk level**. Self-balancing guarantees O(log n) lookups for the highest-risk district.

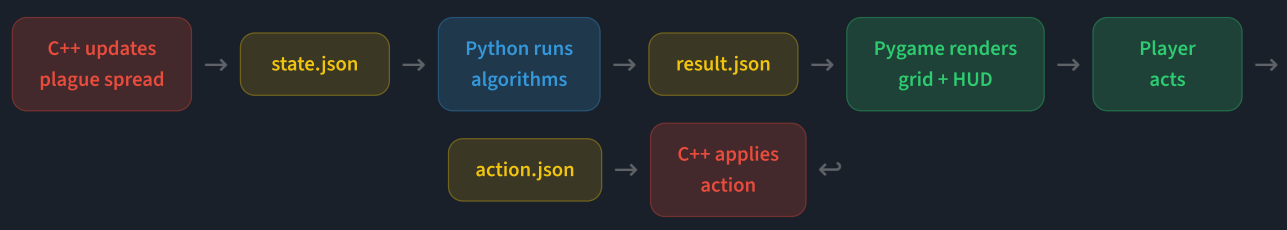
Why AVL over BST? A standard BST can degrade to O(n) if risk values are inserted in order. AVL's rotations keep it balanced at O(log n).

4 System Architecture & JSON Bridge

Three layers communicate through **JSON files** — no direct bindings between C++ and Python. Each component reads and writes specific files in a sequential cycle.



Turn Cycle



> **Why JSON instead of direct bindings?** Human-readable, debuggable, decouples components, and both languages support it natively (`nlohmann/json` in C++, `json` module in Python).

5 JSON Schema

state.json C++ writes

```
// Full game snapshot { "turn": 3, "grid_size":
8, "districts": [ {"row":0, "col":1, "risk":8,
"state":"infected"} ], "infection_chain": [
{"row":3,"col":4, "turn_infected":1} ],
"stats": { "total_infected": 4,
"total_healthy": 52 } }
```

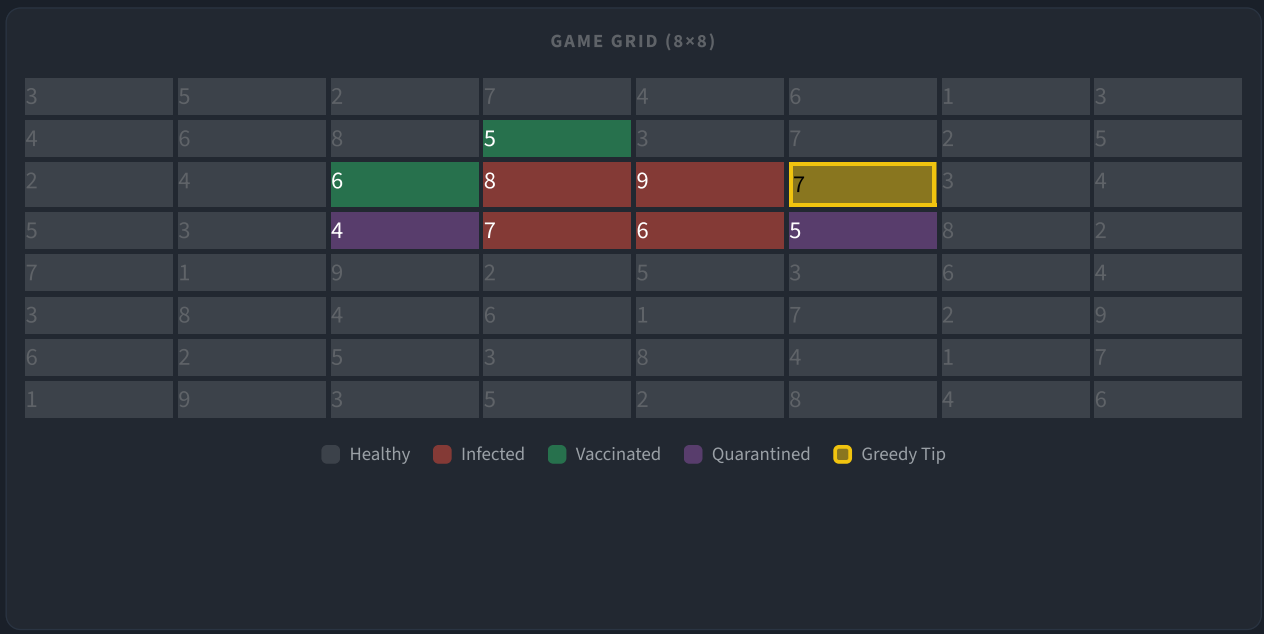
result.json Python writes

```
{ "greedy_recommendation": { "row":2, "col":5,
"risk":9 }, "quarantine_plan": { "found": true,
"districts": [ {"row":2,"col":3},
{"row":4,"col":5} ], "cost": 4 } }
```

action.json Pygame writes

```
{ "turn": 3, "action_type": "vaccinate",
"target": { "row":2, "col":5} }
```

6 Pygame Interface Wireframe



Interaction: Player clicks a grid cell to select it → clicks VACCINATE to confirm, or clicks APPLY QUARANTINE to accept the backtracking plan, or SKIP TURN to pass. Each action generates an `action.json` file read by the C++ engine.